GCA – Session 01

Geavanceerde Computerarchitectuur

Joris Putteneers
joris.putteneers@kuleuven.be

12-11-2026 (academic year 2026/2027)

Contents

1 Introduction **2**
1.1 Goals, learning outcomes, and expectations . . . 2
1.2 Lab sessions . . . 3
1.3 Responsibilities and evaluation . . . 3

2 Project **3**
2.1 Goal and organization . . . 3
2.2 Implementation . . . 3
2.3 Some topic examples . . . 4
2.4 Grading – guidelines based on thesis evaluation matrix . . . 4
2.5 Important dates . . . 4

3 Reading **4**

4 Where to run? **5**
4.1 On the lab PCs (no GPU of your own) . . . 5
4.2 On your own laptop: Windows with WSL . . . 5
4.3 On your own laptop: Linux (Ubuntu 24.04) . . . 6

5 Examples: from “hello” to flipping an array **7**
5.1 01_hello.cu . . . 7
5.2 02_threads.cu . . . 7
5.3 02b_threads_2d.cu . . . 8
5.4 03_blocks.cu . . . 9
5.5 04_memory.cu . . . 10
5.6 05_bounds.cu . . . 12

5.7 06_flip.cu 12
5.8 07_flip_inplace.cu 14

6 Expected for next week **15**

6.1 08_flip_2d.cu: flip a 2D array (mirror an image horizontally) 15

7 Cheat sheet **16**

1 Introduction

1.1 Goals, learning outcomes, and expectations

The goal of this course is to introduce students to the fundamentals of general purpose GPU programming. We shall dive into the concepts of parallel multi-thread processing, learning how to leverage the large number of GPU cores and how to efficiently use the device's memory. Among others, concepts such as atomic operation, synchronous multi-threading, and dynamic parallelism are explored during the course. In addition to guided exercises, the students will partake in a short project, tackling a research or programming challenge of their liking using the newly acquired knowledge. By the end, the students are foreseen to have the necessary understanding of GPUs and parallel programming that will allow them to utilize GPUs in their future work. Throughout the course NVIDIA GPUs and C++ are employed (CUDA API).

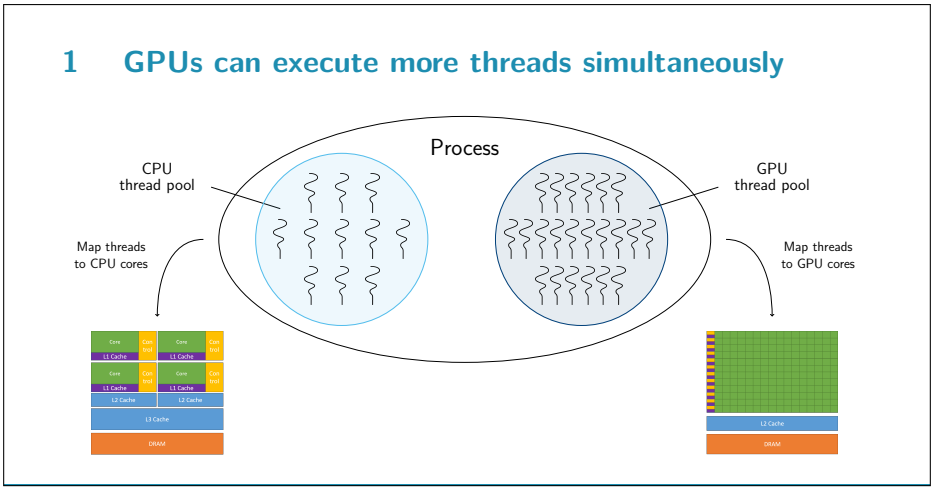


Figure 1: GPUs can execute more threads simultaneously than CPUs.

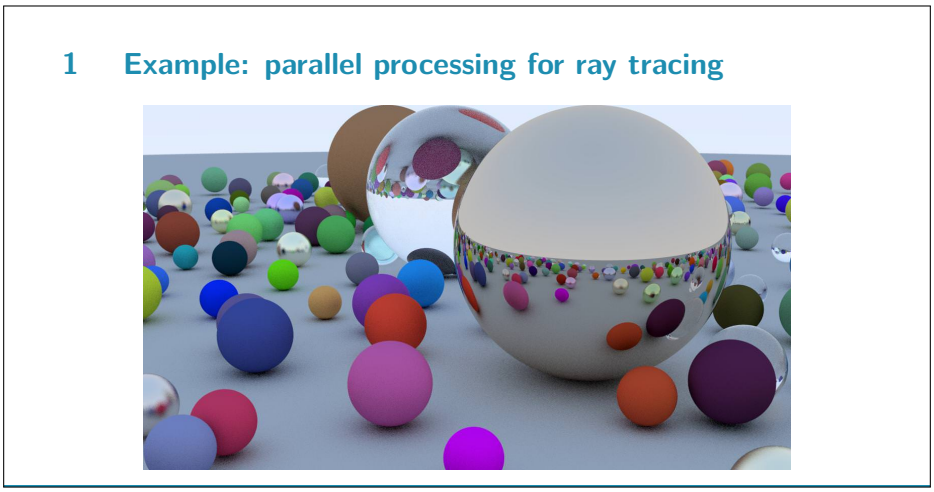


Figure 2: Example: parallel processing for ray tracing.

1 CUDA, an API for programming Nvidia GPUs

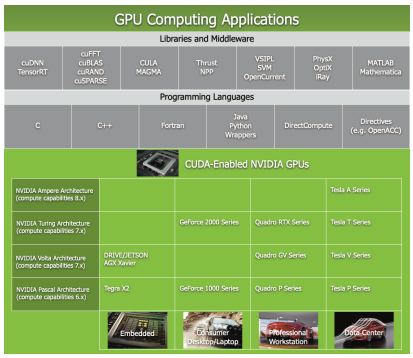


Figure 3: CUDA, an API for programming NVIDIA GPUs.

1.2 Lab sessions

The below sections outline the five guided lab sessions (each 3h).

Organization — 11 sessions in total:

- 5 lab sessions
- 6 project sessions (5 project, 1 presentations)

1.3 Responsibilities and evaluation

Lab sessions: 50%

- 5 single-page reports, one per lab session

Project: 50%

- Project report
- Project presentation
- Code release

2 Project

2.1 Goal and organization

- The goal of the project is to tackle a computational challenge using the GPU.
- This should be something that aligns with your interests – to support your thesis or another lab; to implement something for a hobby / private project; to boost your portfolio and get a better job.
- We will allocate 5 lab sessions to development and 1 session to the presentations.

2.2 Implementation

- Use the concepts we have gotten acquainted with so far and feel free to explore other paradigms. You can use another programming language and also build on another library – keep in mind that you should still understand and describe the intricacies of the implementation.

2.3 Some topic examples

In case you don't know where to start, here are a few biased examples:

- An RF **ray tracer** to determine how Wi-Fi waves propagate in (and outside of) your kot.
- An **optimization algorithm** to determine where to put the Wi-Fi router for optimal coverage (considering either only yourself or all inhabitants).
- **Data compression** on an edge device that collects sensor samples and analyzes/compresses/-transforms these in order to send the information over a low-throughput link.
- Real-time **statistical analysis** of a video stream.

2.4 Grading – guidelines based on thesis evaluation matrix

Implementation

- The implementation works, while clearly addressing the tackled problem
- Includes the concepts introduced in the lab or sufficiently motivates other approaches

Report (no min/max length – try to be concise)

- Clearly describes the problem and motivates the usage of the GPU
- Introduces the implementation and methods in detail (includes references to the repository or code listings)
- Highlights the shortcomings and possible improvements

Presentation (up to 10 min + 10 min for Q&A)

- Clearly describes the problem and motivates the usage of the GPU
- Provides a high-level overview of the implementation
- Highlights the shortcomings and possible improvements

2.5 Important dates

Please submit the reports on time, so we can close off the lab on the day of the presentations (focus on the thesis / exams afterwards).

3 Reading

Read the first chapter of the CUDA Programming Guide:
https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/introduction.html

With focus on:

- CUDA and the CUDA Programming Guide
- 1.1.1. The Graphics Processing Unit
- 1.1.2. The Benefits of Using GPUs
- 1.1.3. Getting Started Quickly
- 1.2.2.1. Thread Blocks and Grids
- 1.2.2.1.1. Thread Block Clusters
- 1.2.2.2. Warps and SIMT

- 1.2.3.4. Unified Memory

4 Where to run?

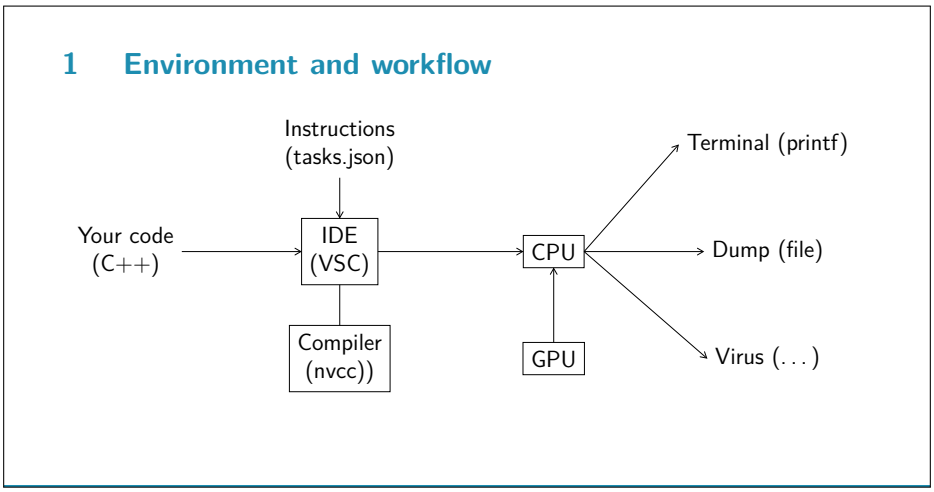


Figure 4: Environment and workflow.

4.1 On the lab PCs (no GPU of your own)

1. Use a desktop with a grey/silver front: these have an NVIDIA GPU (from next week).

GPU:	NVIDIA GeForce GT 730
Compute capability:	3.5
Memory:	1024 MB
SMs:	2 (192 cores per SM, 384 cores total)

2. Boot into Linux (not Windows).

3. Log in:

   user: `student`
   password: `studentXX` (XX = the PC number)

   Admin password (only if something needs to be changed): ask the teacher.

4. Test:

```
nvcc -arch=sm_35 -o hello hello.cu && ./hello
```

4.2 On your own laptop: Windows with WSL

1. Windows: check the laptop has an NVIDIA GPU, then install the latest NVIDIA driver from nvidia.com. Do **not** install any Linux driver inside WSL.

2. PowerShell (admin):

```
wsl --install
```

   Restart, open Ubuntu, create a user.

3. Ubuntu:

```
nvidia-smi
sudo apt update && sudo apt install -y build-essential
wget https://developer.download.nvidia.com/compute/cuda/repos/wsl-ubuntu/x86_64/cuda-keyring_1.1-1_all.
    ↪ deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update && sudo apt install -y cuda-toolkit
echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc && source ~/.bashrc
```

`nvidia-smi` must show the GPU.
GTX 10-series (Pascal) laptops: use `cuda-toolkit-12-9` instead of `cuda-toolkit`.

4. VS Code: install on Windows + the "WSL" extension. Then in Ubuntu:

```
mkdir ~/gca && cd ~/gca && code .
```

Put the course `tasks.json` in `~/gca/.vscode/`

5. Test:

```
nvcc -o hello hello.cu && ./hello
```

4.3 On your own laptop: Linux (Ubuntu 24.04)

1. Driver:

```
sudo ubuntu-drivers install
sudo reboot
nvidia-smi
```

`nvidia-smi` must show the GPU. Top right it shows "CUDA Version": the newest CUDA this driver supports.

2. Toolkit:

```
sudo apt update && sudo apt install -y build-essential
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64/cuda-keyring_1.1-1_all.
    ↪ deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update && sudo apt install -y cuda-toolkit
echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc && source ~/.bashrc
```

- Install only `cuda-toolkit` (not `cuda`: that also installs a second driver).
- The toolkit version (`nvcc -version`) must not be newer than the "CUDA Version" in `nvidia-smi`. If it is, or on GTX 10-series (Pascal) GPUs: use `cuda-toolkit-12-9` instead.
- Ubuntu 22.04: replace `ubuntu2404` with `ubuntu2204` in the `wget` line.

Simplest alternative (older CUDA 12.0, no PATH step needed):

```
sudo apt install -y nvidia-cuda-toolkit
```

3. VS Code:

```
sudo snap install code --classic
mkdir ~/gca && cd ~/gca && code .
```

Install the C/C++ extension. Put the course `tasks.json` in `~/gca/.vscode/`

4. Test:

```
nvcc -o hello hello.cu && ./hello
```

5 Examples: from “hello” to flipping an array

5.1 01_hello.cu

```cpp
// <cstdio> is the C standard input/output library (C++ name for stdio.h).
// We need it for printf on the CPU. On the GPU, CUDA provides printf itself.
#include <cstdio>

// __global__ marks a kernel: it is CALLED from the CPU but RUNS on the GPU.
// "global" because it is visible to both sides. The only way to run your own code
// on the GPU is through a kernel. Helpers marked __device__ can then be called from a kernel.
// void: -> A kernel MUST return void; results are written to memory.
__global__ void hello() {
    // this printf runs on the GPU and is NOT the stdio one. It has the same name and the buffer in GPU memory .
    printf("Hello from the GPU\n");
}

// main() ALWAYS runs on the CPU. Only the body of hello() runs on the GPU.
int main() {
    // Launch the kernel on the GPU: <<<number of blocks, threads per block>>>.
    // " launch the hello kernel with 1 block of 1 thread".
    // Each thread runs the whole body of hello() once, so 1 thread = 1 line of output.
    hello<<<1, 1>>>();
    // hello<<<2, 8>>>() would be 2 blocks of 8 threads = 16 threads which would be 16 times "Hello from the GPU".
    // The launch is asynchronous: the CPU does not wait and continues immediately.

    // cudaDeviceSynchronize() is a CUDA function, called on the CPU, that makes the CPU
    // wait until the GPU has finished all launched work. It also flushes the GPU's printf output.
    // Then the CPU simply continues with the next line, as it always would.
    // Without it, the program could end before the GPU has printed anything.
    // cudaDeviceSynchronize = wait for the GPU
    cudaDeviceSynchronize();


    // this is the normal C printf from <cstdio>, running on the CPU and writing to the terminal.
    printf("Hello from the CPU\n");
}

// Output:
// Hello from the GPU
// Hello from the CPU
```

5.2 02_threads.cu

```cpp
#include <cstdio>

__global__ void hello() {
    // threadIdx.x is a built-in variable that exists automatically in every kernel.
    // It is the index of this thread within its block: 0..7 here.
    // Every thread sees a different value; that is how threads tell themselves apart.
    //
    // threads each run this line once, so you get 8 lines, 0 to 7.
    //
    // %d is a placeholder: printf replaces it with the integer argument (threadIdx.x).
    // \n is a newline.
    // (std::cout does not work on the GPU, only printf.)
    //
    // threadIdx is ONE variable, a struct with three fields: threadIdx.x, threadIdx.y, threadIdx.z.
    // (There is no "threadIdy"; it is always threadIdx.something.)
    // With a 1D launch like <<<1, 8>>>, only .x changes; .y and .z are always 0.
    printf("thread %d\n", threadIdx.x);
}

int main() {
    hello<<<1, 8>>>();
    cudaDeviceSynchronize();
}
```

```
24
25 // Output (order is not guaranteed, but within one block it is usually in order):
26 // thread 0
27 // thread 1
28 // thread 2
29 // thread 3
30 // thread 4
31 // thread 5
32 // thread 6
33 // thread 7
34 //
35 // Why not guaranteed: all threads run in parallel, and the GPU's hardware scheduler decides
36 // which threads execute when. Threads are run in groups of 32 (warps). Within one warp the
37 // printf usually comes out in thread order, but CUDA does not promise it. Between warps and
38 // between blocks there is no order at all: blocks can run on different parts of the GPU, at the
39 // same time, in any order.
40 // never write code that depends on thread order!
```

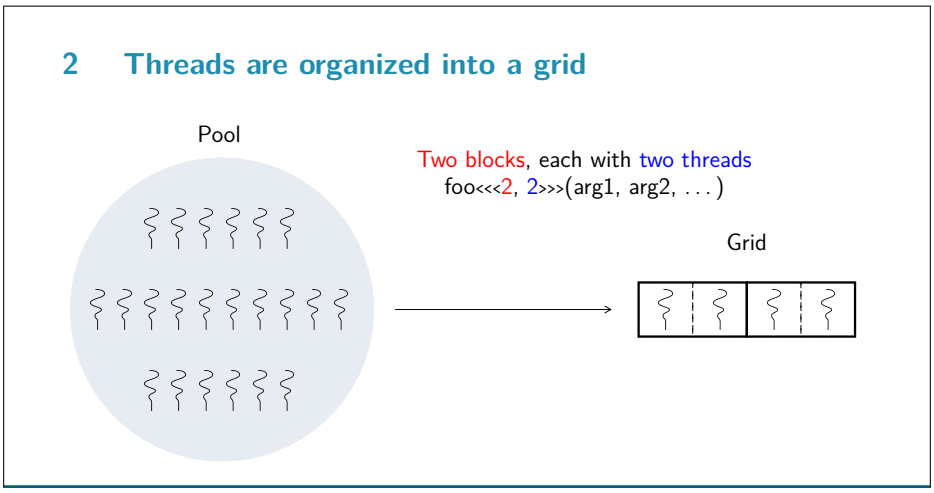


Figure 5: Threads are organized into a grid of blocks.

5.3 02b_threads_2d.cu

```
1  // Why 2D/3D? It is purely a convenience for the programmer, not a hardware feature.
2  // The hardware still sees the threads as one long line (x counts fastest, then y, then z)
3  // and groups them into warps of 32 in that order. Performance is the same.
4  // The benefit: the indices match the shape of the data.
5  //   - 2D: images (x = column, y = row), matrices.  3D: volumes (CT scans, voxels, 3D simulations).
6  //   - Without it you would compute row = i / W and col = i % W yourself; with 2D you get them directly.
7  // Very common: almost every image or matrix kernel uses 2D blocks, e.g. 16 x 16 threads.
8  // Every CUDA GPU supports 1D, 2D and 3D blocks and grids (block z is limited to 64).
9  #include <cstdio>
10
11 __global__ void hello() {
12     // Now the block is 2D, so every thread has two indices:
13     //   threadIdx.x = column inside the block: 0..3
14     //   threadIdx.y = row inside the block:    0..1
15     printf("thread x=%d y=%d\n", threadIdx.x, threadIdx.y);
16 }
17
18 int main() {
19     // dim3 is a CUDA type with three numbers (x, y, z); missing ones default to 1.
20     // dim3 threads(4, 2) = a block of 4 columns by 2 rows = 8 threads.
21     // Writing <<<1, 8>>> is shorthand for dim3(1, 1, 1) blocks and dim3(8, 1, 1) threads.
22     // The grid can be 2D in the same way: dim3 blocks(2, 3) gives blockIdx.x and blockIdx.y.
23     // 2D is convenient for images and matrices: x = column, y = row.
24     // dim3 is a CUDA TYPE: a small struct with three numbers x, y, z (like int, but with three values).
25     // "threads"is just the name of our variable;
26     // The built-ins are only threadIdx, blockIdx, blockDim and gridDim.
27     dim3 threads(4, 2);
28     hello<<<1, threads>>>();
```

```
    cudaDeviceSynchronize();
}

// Output (same 8 threads as 02, but now each has an x and a y; order not guaranteed):
// thread x=0 y=0
// thread x=1 y=0
// thread x=2 y=0
// thread x=3 y=0
// thread x=0 y=1
// thread x=1 y=1
// thread x=2 y=1
// thread x=3 y=1
```

5.4 03_blocks.cu

```
#include <cstdio>

__global__ void hello() {
    // Also built-in variables:
    // blockIdx.x = index of this block in the grid: 0..1 here
    // blockDim.x = threads per block, i.e. the 2nd launch number: 4 here
    // gridDim.x  = number of blocks, i.e. the 1st launch number: 2 here
    //
    // to get a unique global index: block 1, thread 2 -> 1*4+2 = 6.
    // global index = blockIdx.x * blockDim.x + threadIdx.x
    //
    // Block 0 covers indices 0..3, so block 1 starts after those 4: at index 4.
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    printf("block %d thread %d -> global index %d\n", blockIdx.x, threadIdx.x, i);
}

int main() {
    hello<<<2, 4>>>();
    cudaDeviceSynchronize();
}

// Output (block 1 may print before block 0; that differs between runs):
// block 0 thread 0 -> global index 0
// block 0 thread 1 -> global index 1
// block 0 thread 2 -> global index 2
// block 0 thread 3 -> global index 3
// block 1 thread 0 -> global index 4
// block 1 thread 1 -> global index 5
// block 1 thread 2 -> global index 6
// block 1 thread 3 -> global index 7  e.g. (1 * 4) + 3
```

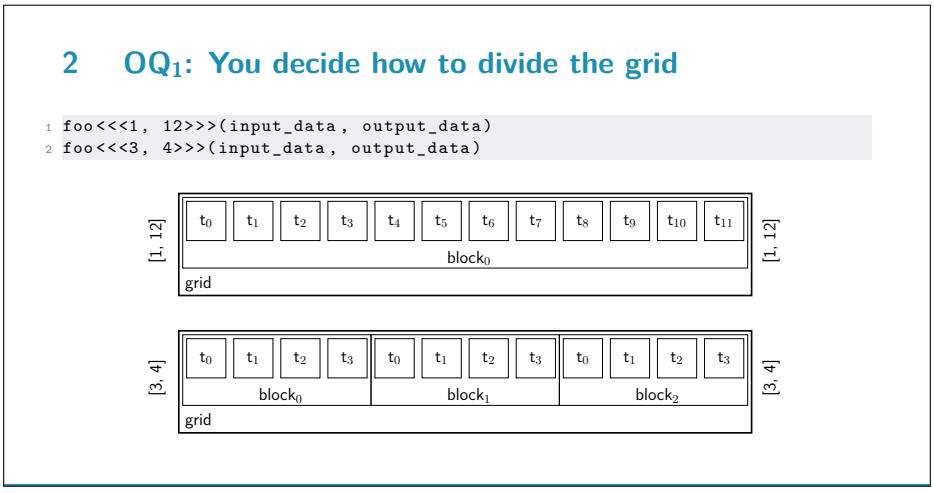


Figure 6: You decide how to divide the grid.

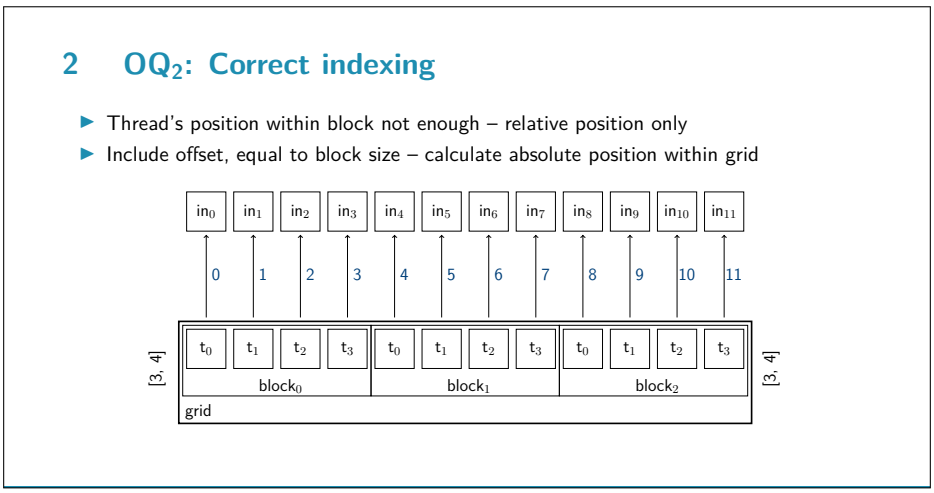


Figure 7: Correct indexing: add the block offset to get the absolute position.

5.5 04_memory.cu

```
//  Every line in main() runs on the CPU; only the kernel fill() runs on the GPU.
//   1. int* a;                                  (CPU) Create an empty pointer. It points nowhere yet.
//   2. cudaMallocManaged(&a, N * sizeof(int));  (CPU calls it) Reserve 32 bytes that both CPU and GPU
//                                               can reach, and put the address into a. The memory
//                                               exists now, but holds no useful values yet.
//   3. fill<<<2, 4>>>(a);                       (CPU starts it, GPU runs it) 8 GPU threads each write
//                                               their own index into their own element: 0 1 2 ... 7.
//                                               The CPU does not wait here.
//   4. cudaDeviceSynchronize();                 (CPU) Wait until the GPU has finished filling.
//   5. for loop with printf                     (CPU) Read the array and print 0 1 2 3 4 5 6 7.
//   6. cudaFree(a);                             (CPU calls it) Give the memory back; we are done with it.
#include <cstdio>

// Preprocessor constant: before compiling, every N in the file is replaced by 8.
// It represents the length of the array.
#define N 8

// A kernel that receives a pointer adress to the array.
// It does not MAKE the array: the memory is reserved in main() by cudaMallocManaged.
// This kernel FILLS the existing array: it writes the values 0..7 into it.
//
// This replaces the CPU loop "for (i = 0; i < N; i++) a[i] = i;" -- the threads ARE the loop.
__global__ void fill(int* a) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // No bounds check needed HERE because N = 8 and we launch exactly 2 * 4 = 8 threads:
    a[i] = i;
}

int main() {
    // a is a pointer: a variable that holds a memory address (the start of an int array).
    // It does not point anywhere yet.
    int* a;

    // Allocate N * sizeof(int) = 8 * 4 = 32 bytes of unified memory, (a property of int is that they use 4
    ↪ bytes)
    // reachable by both the CPU and the GPU.
    // &a is the address of the pointer itself, so the function can write the new address into a.
    cudaMallocManaged(&a, N * sizeof(int));

    fill<<<2, 4>>>(a);

    // The CPU waits until the kernel is done. Only then is it safe to read a on the CPU.
    cudaDeviceSynchronize();

    for (int i = 0; i < N; i++) printf("%d ", a[i]);
    printf("\n");

    // Release the memory from cudaMallocManaged (the CUDA equivalent of free()).
```

```
48      cudaFree(a);
49  }
50
51  // Output:
52  // 0 1 2 3 4 5 6 7
```

Why we need bounds. The number of threads you launch is always blocks $\times$ threads per block, so it's always a multiple of the block size. The size of your data (N) is set by the problem and almost never matches. The bounds check `if (i < N)` stops the extra threads from reading or writing memory outside the array. Without it, the program can silently corrupt memory or crash, even when the output looks correct.

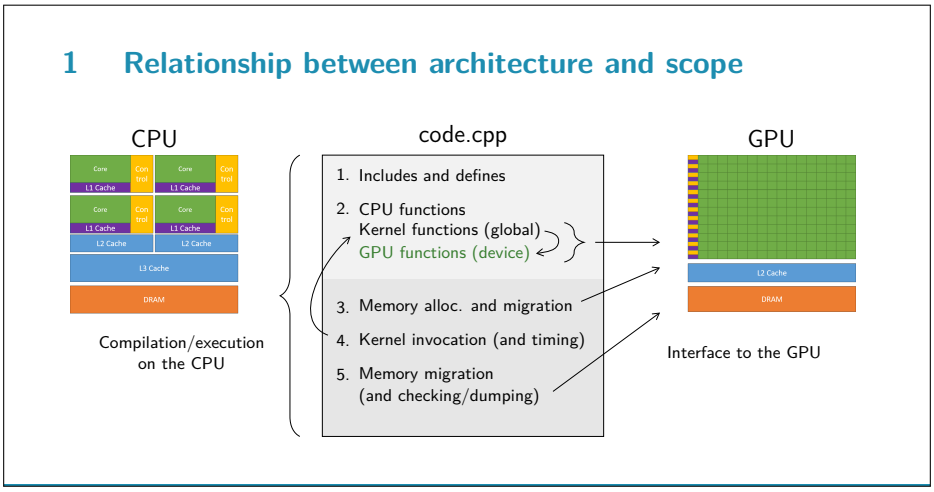


Figure 8: Which part of the program runs on the CPU and which on the GPU.

5.6 05_bounds.cu

```cpp
#include <cstdio>

#define N 10

__global__ void fill(int* a, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // We launch 3 blocks * 4 threads = 12 threads, but the array has only 10 elements.
    // Threads 10 and 11 must do nothing, otherwise they write outside the array.
    if (i < n) a[i] = i;
}

int main() {
    int* a;
    cudaMallocManaged(&a, N * sizeof(int));

    int threads = 4;
    // Integer division rounds DOWN: 10 / 4 = 2 blocks = 8 threads, which is too few.
    // Adding (threads - 1) makes it round UP: (10 + 3) / 4 = 3 blocks = 12 threads.
    int blocks = (N + threads - 1) / threads;

    // N is the #define above (10). We pass it to the kernel as parameter n,
    // because the kernel needs the array length for its bounds check.
    fill<<<blocks, threads>>>(a, N);
    cudaDeviceSynchronize();

    for (int i = 0; i < N; i++) printf("%d ", a[i]);
    printf("\n");

    cudaFree(a);
}

// Output (threads 10 and 11 exist but write nothing):
// 0 1 2 3 4 5 6 7 8 9
```

5.7 06_flip.cu

```cpp
// The plan: the CPU makes two arrays and fills "in" with 0 1 2 ... 8.
// The GPU copies every element of "in" to its mirrored position in "out".
// The CPU then prints both arrays and checks the result.
//
// Steps in main():
//   1. Reserve two arrays of 9 ints: in and out.
//   2. Fill in on the CPU with 0 ... 8.
//   3. Choose the launch: 4 threads per block, (9 + 3) / 4 = 3 blocks = 12 threads.
//   4. Launch flip: threads 0..8 each move one element; threads 9..11 are stopped by the bounds check.
//   5. Wait for the GPU.
//   6. Print in and out on the CPU.
//   7. Verify: in[i] was i, so out[i] must be N - 1 - i.
//   8. Free both arrays.
//
// About step 3 -- warps:
//   The GPU never runs a single thread on its own. It runs threads in groups of 32, called a warp.
//   All 32 threads of a warp execute the same instruction at the same time.
//   Every block is cut into warps:
//     block of 4 threads   -> 1 warp, of which only 4 lanes work and 28 sit idle
//     block of 256 threads -> 8 full warps, nothing wasted
//   So in real code the block size is a multiple of 32 (typically 256).
//   Here we use 4 only to keep the example and its output small.
//
//   Real code example, for one million elements:
//     #define N 1000000
//     int threads = 256;
//     int blocks = (N + threads - 1) / threads;   -> 3907 blocks
//   3907 * 256 = 1,000,192 threads; the last 192 are stopped by the bounds check.
//   (With N that large you would not print the arrays; the verification loop does the checking.)
//
// Why two arrays: every thread reads from one array and writes to another, so no thread can
// overwrite a value that another thread still needs. The order in which threads run does not matter.

```

```cpp
#include <cstdio>

#define N 9

// in  = pointer to the input array (0 1 2 ... 8). const = the kernel promises to only read it.
// out = pointer to the output array; the kernel writes the flipped values here.
// n   = number of elements (9), needed for the mirror position and the bounds check.
__global__ void flip(const int* in, int* out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // this is the flipping logic: each thread puts its element at the mirrored position.
    if (i < n) out[n - 1 - i] = in[i];
}

int main() {
    int *in, *out;
    cudaMallocManaged(&in, N * sizeof(int));
    cudaMallocManaged(&out, N * sizeof(int));

    // The starting array is made here: the cudaMallocManaged above reserves the memory,
    // and this loop fills it on the CPU with 0 1 2 ... 8.
    for (int i = 0; i < N; i++) in[i] = i;

    int threads = 4;
    int blocks = (N + threads - 1) / threads; //3
    flip<<<blocks, threads>>>(in, out, N);
    cudaDeviceSynchronize();

    printf("in:  ");
    for (int i = 0; i < N; i++) printf("%d ", in[i]);
    printf("\nout: ");
    for (int i = 0; i < N; i++) printf("%d ", out[i]);
    printf("\n");

    // check
    for (int i = 0; i < N; i++) {
        if (out[i] != N - 1 - i) {
            printf("error at %d\n", i);
            return 1;
        }
    }
    printf("correct\n");

    cudaFree(in);
    cudaFree(out);
}

// Output:
// in:  0 1 2 3 4 5 6 7 8
// out: 8 7 6 5 4 3 2 1 0
// correct
```

5.8 07_flip_inplace.cu

```cpp
// In-place is an alternative with one advantage and some drawbacks:
//   + Uses ONE array instead of two: half the memory.
//   - Not faster: both versions read N values and write N values in total.
//   - Easier to get wrong: if all N threads simply copy their element to the mirror position,
//     a thread can overwrite a value that another thread has not read yet (a race condition,
//     topic of session 2). That is why only the first half of the threads work, each swapping one pair.
//   - The original input is gone afterwards, so you cannot compare against it.
// It is included mainly to show a race-condition trap.
//
// What is a race condition?
//   Two or more threads access the SAME memory location, at least one of them WRITES,
//   and nothing controls which one goes first. The result then depends on timing: on which
//   thread happens to be faster. It can be right in one run and wrong in the next.
//   On a GPU, thousands of threads run in parallel in no guaranteed order, so this is easy to cause.
//   Analogy: two people withdraw money from the same account at the same moment. Both read
//   balance 100, both subtract 50, both write back 50. One withdrawal is lost.
//   How to avoid it:
//     - make sure every memory location is written by only one thread (as in 06 and the swap below)
//     - or use synchronization / atomic operations (session 2).
//
// The race condition, concretely. The "obvious" in-place kernel would be:
//
//     if (i < n) a[n - 1 - i] = a[i];      // WRONG: all 9 threads, one array
//
// Thread 0 must write a[0] (= 0) into a[8]. Thread 8 must write a[8] (= 8) into a[0].
// Both touch a[0] and a[8], and nobody decides who goes first:
//   - if thread 8 reads a[8] BEFORE thread 0 overwrites it: a[0] = 8   (correct)
//   - if thread 8 reads a[8] AFTER  thread 0 overwrote it:  a[0] = 0   (wrong: the 8 is lost)
// The same happens for every pair. A possible result is 0 1 2 3 4 3 2 1 0 (a mirror, not a flip),
// and it can differ from run to run. That is what a race condition is: the result depends on
// which thread happens to be faster.
//
// The fix below: only n / 2 threads, and each thread does the WHOLE swap of one pair itself.
// Pair (i, n-1-i) is touched by exactly one thread, so no two threads ever touch the same element.
// tmp is a local variable: every thread has its own private copy (in its own register).

#include <cstdio>

#define N 9

__global__ void flip(int* a, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n / 2) {
        int tmp = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = tmp;
    }
}

int main() {
    int* a;
    cudaMallocManaged(&a, N * sizeof(int));

    for (int i = 0; i < N; i++) a[i] = i;

    int threads = 4;
    int blocks = (N / 2 + threads - 1) / threads;
    flip<<<blocks, threads>>>(a, N);
    cudaDeviceSynchronize();

    for (int i = 0; i < N; i++) printf("%d ", a[i]);
    printf("\n");

    cudaFree(a);
}

// Launch: N / 2 = 4 pairs -> (4 + 3) / 4 = 1 block of 4 threads.
// Thread 0 swaps 0<->8, thread 1 swaps 1<->7, thread 2 swaps 2<->6, thread 3 swaps 3<->5.
// The middle element (4) stays where it is.
// Output:
// 8 7 6 5 4 3 2 1 0
```

6 Expected for next week

A LaTeX-created PDF document with a solution for exercise `08_flip_2d.cu`.

6.1 08_flip_2d.cu: flip a 2D array (mirror an image horizontally)

Problem. A 2D array of W columns by H rows (think: an image). Reverse every row:

in:						out:					
0	1	2	3	4	5	5	4	3	2	1	0
6	7	8	9	10	11	11	10	9	8	7	6

Use a 2D grid of 2D blocks: every thread handles one element and gets an `x` (column) and a `y` (row).

Goal.

- Compute `x` and `y` for each thread.
- Convert `(x, y)` to a position in memory.
- Bounds check in **both** directions.
- Verify the result on the CPU.

7 Cheat sheet

This section lists several functions, definitions, etc., handy for the GCA lab. It is a living document, where updates are added according to the lab sessions and feedback from the students. Thanks for pointing things out.

GPU memory allocation

```
float* d_A;
cudaMalloc(&d_A, size);
```

Memory migration to / from GPU

```
float* h_A = (float*)malloc(size);
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(h_A, d_A, size, cudaMemcpyDeviceToHost);
```

Kernel invocation (100 blocks, with 80 threads each)

```
foo<<<100, 80>>>(d_A);
```

Thread indexing

```
int i = threadIdx.x;
```

Setting to global scope (invoked by CPU or GPU)

```
__global__ void foo(d_A){...}
```

Setting to GPU scope (invoked by GPU)

```
__device__ float foo(d_A){...}
```

Error checking (after a kernel launch)

```
foo<<<100, 80>>>(d_A);
cudaError_t err = cudaGetLastError();
if (err != cudaSuccess) printf("launch error: %s\n", cudaGetErrorString(err));
err = cudaDeviceSynchronize();
if (err != cudaSuccess) printf("kernel error: %s\n", cudaGetErrorString(err));
```

Timing (CPU)

```
#include <chrono>
const auto start = std::chrono::steady_clock::now();
...
const auto end = std::chrono::steady_clock::now();
const std::chrono::duration<double> elapsed_seconds{end - start};
```

Timing (GPU)

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
cudaEventRecord(start);
...
cudaEventSynchronize();
cudaEventRecord(stop);
float milliseconds;
cudaEventElapsedTime(&milliseconds, start_cuda, stop_cuda);
```